Assignment 4 Facade Pattern

Identification

Assignment title: Facade Pattern

Full name: Yermekkyzy Assel

Group: SE-2524

Topic: Option A - Media/File Conversion Facade

GitHub repository link: https://github.com/yerassell/assignment4-facade-pattern.git

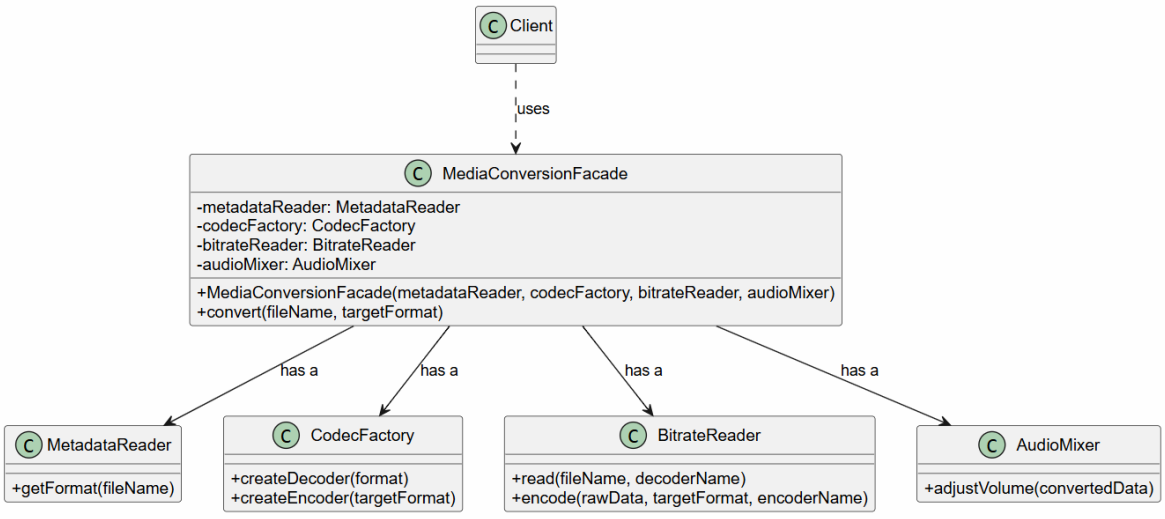
Last commit: 220f3f35b34ed22461c91224584a2d8d3069b290

Introduction

I chose the topic of Media Conversion Facade. This is an application that converts a file into a target format. This conversion includes reading the file's metadata, selecting the correct decoder and encoder, reading raw audio samples, adjusting the volume and encoding the samples into the new format. At the end the new file of different format is saved.

The Facade pattern provides a class with one simple method which converts a file into a target format. This hides the complexity and execution steps leaving the client decoupled. Without the Facade pattern the client would be calling all steps manually and would be tightly coupled.

UML class diagram



Picture 1. UML class diagram

Clean Code

1. *Meaningful names that make the Facade's simplified API self-explanatory*
   Explanation:
   The names used in the code are self-explanatory. The user immediately understands that the variables represent subsystem classes, the constructor assigns those dependencies, and the method performs the conversion
   Excerpt (MediaConversionFacade.java):

```
private MetadataReader metadataReader;
…

public MediaConversionFacade(MetadataReader metadataReader, CodecFactory
codecFactory, BitrateReader bitrateReader, AudioMixer audioMixer) {
    …
}

public void convert(String fileName, String targetFormat) {
    …
}
```

2. *Single Responsibility: the Facade coordinates, it does not reimplement subsystem logic*
   Explanation:
   The Facade does not know how to decode a format or how to adjust the volume. It only knows the correct order of operations. Thus, it coordinates and delegates the actual work to subsystem classes
   Excerpt (MediaConversionFacade.java):

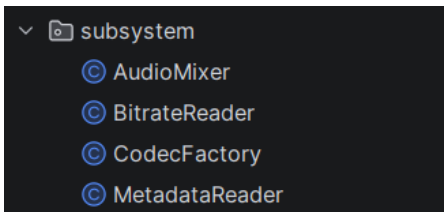
```
String sourceFormat = metadataReader.getFormat(fileName);

String decoderName = codecFactory.createDecoder(sourceFormat);
String encoderName = codecFactory.createEncoder(targetFormat);
```

3. *No leaking of subsystem details through the Facade's public interface*
   Explanation:
   The Client interacts only with the Facade. It does not need to know each subsystem that exist or the way they interact. The Client only passes a file name and a target format.
   Excerpt (Client.java):

```
MediaConversionFacade facade = new MediaConversionFacade(meta, codec,
bitrate, mixer);
facade.convert("song.mp3", "wav");
facade.convert("interview.wav", "mp3");
```

4. *Small, focused subsystem classes with clear responsibilities*
   Explanation:
   Each Subsystem has a specific job. MetadataReader reads format, CodecFactory selects the decoder and encoder, BitrateReader reads raw audio sample and encodes to target format and AudioMixer adjusts the volume
   Excerpt:



Picture 2. Subsystem structure

5. *Proper error handling for subsystem coordination failures*
   Explanation:
   The methods return String data rather than just printing text using void methods. This ensures that the data is actually passed from one step to the next. Another error handling method is returning “Unknown Decoder” instead of crashing which allows Facade to output the final result accurately
   Excerpt (CodecFactory.java):

```
if (format.equalsIgnoreCase("mp3")) {
    return "MP3 Decoder";
} else if (format.equalsIgnoreCase("wav")) {
    return "WAV Decoder";
}
return "Unknown Decoder";
```

Conclusion

What the Facade hides:

The Facade hides the existence of subsystem classes from the Client. It hides the strict sequence of operations to convert a file. The Client does not need to know each step that subsystem performs or their order.

What remains directly accessible:

The Client is still able to call a method of specific subclass directly if needed. The subsystem classes are unaware of the Facade but they remain accessible by their own.